

MATH00041: Introduction to Programming

NumPy - II

Lecture 10

Archontis Giannakidis, PhD

Outline

- **Vectorisation**
- **Ufunc**
- **log**
- **Summation**
- **Trigonometry**
- **Vector**
- **Matrix**

NumPy ufuncs

What are ufuncs?

NumPy's **ufuncs** are "Universal Functions" that operate on **ndarrays**.

Why use ufuncs?

NumPy's ufuncs enable faster computation by implementing **vectorisation** and providing broadcasting, alongside reduce, accumulate and other useful methods. ufuncs also take additional arguments, like:

where boolean array or condition defining where the operations should take place.

dtype defining the return type of elements.

out output array where the return value should be copied.

What is Vectorisation?

Converting iterative statements into a vector-based operation is called vectorisation.

It is faster as modern CPUs are optimised for such operations.

Add the Elements of Two Lists

list 1: [1, 2, 3, 4]

list 2: [4, 5, 6, 7]

One way of doing it is to iterate over both of the lists and then sum each element.

Example:

Without ufunc, we can use Python's built-in `zip()` method:

```
x = [1, 2, 3, 4]
y = [4, 5, 6, 7]
z = []
```

```
for i, j in zip(x, y):
    z.append(i + j)
print(z)
```

```
>>>
```

```
[5, 7, 9, 11]
```

NumPy has a ufunc for this, called `add(x, y)` that will produce the same result.

Example:

With ufunc, we can use the `add()` function:

```
import numpy as np
```

```
x = [1, 2, 3, 4]
```

```
y = [4, 5, 6, 7]
```

```
z = np.add(x, y)
```

```
print(z)
```

```
>>>
```

```
[5, 7, 9, 11]
```

How To Create Your Own ufunc

To create a **ufunc** in **NumPy**, define a function and add it to the **ufunc** library using **frompyfunc()**.

1. *function* - the name of the function.
2. *inputs* - the number of input arguments (arrays).
3. *outputs* - the number of output arrays.

Example:

Create your own ufunc for addition:

```
import numpy as np

def myadd(x, y):
    return x+y

myadd = np.frompyfunc(myadd, 2, 1)

print(myadd([1, 2, 3, 4], [5, 6, 7, 8]))
>>>
[6 8 10 12]
```

Check if a Function is a ufunc

Check the *type* of a function to check if it is a ufunc or not.

A ufunc should return `<class 'numpy.ufunc'>`.

Example:

Check if a function is a ufunc:

```
import numpy as np
```

```
print(type(np.add))
```

```
>>>
```

```
<class 'numpy.ufunc'>
```

If it is not a ufunc, it will return another type, like this built-in NumPy function for joining two or more arrays:

Example:

Check the type of another function: concatenate():

```
import numpy as np
```

```
print(type(np.concatenate))
```

```
>>>
```

```
<class 'builtin_function_or_method'>
```

Simple Arithmetic

You could use arithmetic operators $+$ $-$ $*$ $/$ directly between NumPy arrays, but this section discusses an extension of the same where we have functions that can take any array-like objects e.g. lists, tuples etc. and perform arithmetic *conditionally*.

Arithmetic Conditionally: means that we can define conditions where the arithmetic operation should happen. All of the discussed arithmetic functions take a **where** parameter in which we can specify that condition.

Addition

The `add()` function sums the content of two arrays and return the results in a new array.

Example:

Add the values in arr1 to the values in arr2:

```
import numpy as np

arr1 = np.array([10, 11, 12, 13, 14, 15])
arr2 = np.array([20, 21, 22, 23, 24, 25])

newarr = np.add(arr1, arr2)

print(newarr)

>>>
[30 32 34 36 38 40]
```

The example above will return `[30 32 34 36 38 40]`, which is the sums of `10+20`, `11+21`, `12+22` etc.

Subtraction

The `subtract()` function subtracts the values from one array with the values from another array and return the results in a new array.

Example:

Subtract the values in arr2 from the values in arr1:

```
import numpy as np

arr1 = np.array([10, 20, 30, 40, 50, 60])
arr2 = np.array([20, 21, 22, 23, 24, 25])

newarr = np.subtract(arr1, arr2)

print(newarr)

>>> [-10 -1  8 17 26 35]
```

The example above will return `[-10 -1 8 17 26 35]` which is the result of `10-20`, `20-21`, `30-22` etc.

Multiplication

The `multiply()` function multiplies the values from one array with the values from another array and return the results in a new array.

Example:

Multiply the values in arr1 with the values in arr2:

```
import numpy as np

arr1 = np.array([10, 20, 30, 40, 50, 60])
arr2 = np.array([20, 21, 22, 23, 24, 25])

newarr = np.multiply(arr1, arr2)

print(newarr)

>>>
[ 200  420  660  920 1200 1500]
```

The example above will return [200 420 660 920 1200 1500], resulting in 10×20 , 20×21 , 30×22 , etc.

Division

The `divide()` function divides the values from one array with the values from another array and returns the results in a new array.

Example:

Divide the values in arr1 with the values in arr2:

```
import numpy as np

arr1 = np.array([10, 20, 30, 40, 50, 60])
arr2 = np.array([3, 5, 10, 8, 2, 33])

newarr = np.divide(arr1, arr2)

print(newarr)

>>>
```

```
[ 3.33333333  4.         3.         5.         25.         1.81818182]
```

The example above will return `[3.33333333 4. 3. 5. 25. 1.81818182]`, which results from $10/3$, $20/5$, $30/10$ etc.

Power

The `power()` function raises the values from the first array to the power of the values of the second array and returns the results in a new array.

Example:

Raise the values in arr1 to the power of values in arr2:

```
import numpy as np

arr1 = np.array([10, 20, 30, 40, 50, 60])
arr2 = np.array([3, 5, 6, 8, 2, 33])

newarr = np.power(arr1, arr2)

print(newarr)

>>>
```

```
[1000      3200000  729000000 6553600000000  2500      0]
```

The example above will return `[1000 3200000 729000000 6553600000000 2500 0]`, which is the result of $10 \times 10 \times 10$, $20 \times 20 \times 20 \times 20 \times 20$, $30 \times 30 \times 30 \times 30 \times 30 \times 30$, etc.

Remainder

Both the `mod()` and the `remainder()` functions return the remainder of the values in the first array corresponding to the values in the second array and return the results in a new array.

Example:

Return the remainders:

```
import numpy as np

arr1 = np.array([10, 20, 30, 40, 50, 60])
arr2 = np.array([3, 7, 9, 8, 2, 33])

newarr = np.mod(arr1, arr2)

print(newarr)

>>>
[ 1  6  3  0  0 27]
```

The example above will return `[1 6 3 0 0 27]`, which is the remainder when you divide 10 with 3 ($10\%3$), 20 with 7 ($20\%7$), 30 with 9 ($30\%9$), etc.

Absolute Values

Both the `absolute()` and the `abs()` functions do the same absolute operation elementwise but we should use `absolute()` to avoid confusion with Python's inbuilt `math.abs()`

Example:

Return the quotient and mod:

```
import numpy as np

arr = np.array([-1, -2, 1, 2, 3, -4])

newarr = np.absolute(arr)

print(newarr)

>>>
[ 1  6  3  0  0 27]
```

Rounding Decimals

There are primarily five ways of rounding off decimals in NumPy:

1. truncation
2. fix
3. rounding
4. floor
5. ceil

Example:

Truncate elements of following array:

```
import numpy as np

arr = np.trunc([-3.1666, 3.6667])

print(arr)

>>>
[-3.  3.]
```

Example:

Same example, using `fix()`:

```
import numpy as np
```

```
arr = np.fix([-3.1666, 3.6667])
```

```
print(arr)
```

```
>>>
```

```
[-3.  3.]
```

Example:

Round off 3.1666 to 2 decimal places:

```
import numpy as np
```

```
arr = np.around(3.1666, 2)
```

```
print(arr)
```

```
>>>
```

```
3.17
```

The `around()` function increments the preceding digit or decimal by 1 if ≥ 5 else, do nothing.

Example:

Floor the elements of following array:

```
import numpy as np

arr = np.floor([-3.1666, 3.6667])

print(arr)

>>>
[-4.  3.]
```

The `floor()` function rounds off decimal to the nearest lower integer.

Example:

Ceil the elements of the following array:

```
import numpy as np

arr = np.ceil([-3.1666, 3.6667])

print(arr)

>>>
[-3.  4.]
```

The `ceil()` function rounds off decimal to the nearest upper integer.

Logs

NumPy offers built-in functions for finding logarithms with base 2, e, and 10. Additionally, we can create a custom ufunc to calculate logarithms with any base. However, in cases where the calculation is not possible, all of the log functions will return -inf or inf for those elements.

Log at Base 2

Use the `log2()` function to perform log at the base 2.

Example:

Find log at base 2 of all elements of following array:

```
import numpy as np
```

```
arr = np.arange(1, 10)
```

```
print(np.log2(arr))
```

```
>>>
```

```
[0.          1.          1.5849625  2.          2.32192809  2.5849625
 2.80735492  3.          3.169925  ]
```

- ❖ **Note:** The `arange(1, 10)` function returns an array with integers starting from 1 (included) to 10 (not included).

Log at Base 10

Use the `log10()` function to perform the log at the base 10.

Example:

Find log at base 10 of all elements of following array:

```
import numpy as np

arr = np.arange(1, 10)

print(np.log10(arr))

>>>
```

```
[0.          0.30103    0.47712125 0.60205999 0.69897    0.77815125
 0.84509804 0.90308999 0.95424251]
```

Log at Any Base

NumPy does not provide any function to take log at any base, so we can use the `frompyfunc()` function along with inbuilt function `math.log()` with two input parameters and one output parameter:

Example:

```
from math import log
import numpy as np

nplog = np.frompyfunc(log, 2, 1)

print(nplog(100, 15))
>>>
1.7005483074552052
```

NumPy Summations

Addition is an arithmetic operation that involves adding two or more numbers together to obtain a total. It's a process that combines two arguments to produce a single value. On the other hand, summation is a mathematical process that involves adding up a sequence of numbers to obtain the total sum. Summation is performed over n elements, where n is a positive integer, and the result is also a single value.

Example:

Add the values in arr1 to the values in arr2:

```
import numpy as np
```

```
arr1 = np.array([1, 2, 3])
```

```
arr2 = np.array([1, 2, 3])
```

```
newarr = np.add(arr1, arr2)
```

```
print(newarr)
```

```
>>>
```

```
[2 4 6]
```

Example:

Sum the values in arr1 and the values in arr2:

```
import numpy as np
```

```
arr1 = np.array([1, 2, 3])
```

```
arr2 = np.array([1, 2, 3])
```

```
newarr = np.sum([arr1, arr2])
```

```
print(newarr)
```

```
>>>
```

```
12
```

Summation Over an Axis

If you specify `axis=1`, NumPy will sum the numbers in each array.

Example:

Perform summation in the following array over 1st axis:

```
import numpy as np
```

```
arr1 = np.array([1, 2, 3])
```

```
arr2 = np.array([1, 2, 3])
```

```
newarr = np.sum([arr1, arr2], axis=1)
```

```
print(newarr)
```

```
>>>
```

```
[6 6]
```

Cumulative Sum

Cumulative sum means partially adding the elements in the array.

E.g. The partial sum of [1, 2, 3, 4] would be [1, 1+2, 1+2+3, 1+2+3+4] = [1, 3, 6, 10].

Perform partial sum with the `cumsum()` function.

Example:

Perform cumulative summation in the following array:

```
import numpy as np
```

```
arr = np.array([1, 2, 3])
```

```
newarr = np.cumsum(arr)
```

```
print(newarr)
```

```
>>>
```

```
[1 3 6]
```

Products

To find the product of the elements in an array, use the `prod()` function.

Example:

Find the product of the elements of this array:

```
import numpy as np

arr = np.array([1, 2, 3, 4])

x = np.prod(arr)

print(x)

>>>
24
```

Returns: 24 because $1*2*3*4 = 24$

Example:

Find the product of the elements of two arrays:

```
import numpy as np
```

```
arr1 = np.array([1, 2, 3, 4])
```

```
arr2 = np.array([5, 6, 7, 8])
```

```
x = np.prod([arr1, arr2])
```

```
print(x)
```

```
>>>
```

```
40320
```

Returns: 40320 because $1*2*3*4*5*6*7*8 = 40320$

Product Over an Axis

If you specify `axis=1`, NumPy will return the product of each array.

Example:

Perform summation in the following array over 1st axis:

```
import numpy as np
```

```
arr1 = np.array([1, 2, 3, 4])
```

```
arr2 = np.array([5, 6, 7, 8])
```

```
newarr = np.prod([arr1, arr2], axis=1)
```

```
print(newarr)
```

```
>>>
```

```
[ 24 1680]
```

Cumulative Product

Cumulative product means taking the product partially.

E.g. The partial product of [1, 2, 3, 4] is [1, 1*2, 1*2*3, 1*2*3*4] = [1, 2, 6, 24]

Perform partial sum with the `cumprod()` function.

Example:

Take cumulative product of all elements for the following array:

```
import numpy as np
```

```
arr = np.array([5, 6, 7, 8])
```

```
newarr = np.cumprod(arr)
```

```
print(newarr)
```

```
>>>
```

```
[ 5  30 210 1680]
```

Differences

A discrete difference means subtracting two successive elements.

E.g. for [1, 2, 3, 4], the discrete difference would be [2-1, 3-2, 4-3] = [1, 1, 1]

To find the discrete difference, use the `diff()` function.

Example:

Compute discrete difference of the following array:

```
import numpy as np
```

```
arr = np.array([10, 15, 25, 5])
```

```
newarr = np.diff(arr)
```

```
print(newarr)
```

```
>>>
```

```
[ 5  10 -20]
```

Returns: [5 10 -20] because 15-10=5, 25-15=10, and 5-25=-20

We can perform this operation repeatedly by giving parameter **n**.

E.g. for [1, 2, 3, 4], the discrete difference with $n = 2$ would be $[2-1, 3-2, 4-3] = [1, 1, 1]$, then, since $n=2$, we will do it once more, with the new result: $[1-1, 1-1] = [0, 0]$

Example:

Compute discrete difference of the following array twice:

```
import numpy as np

arr = np.array([10, 15, 25, 5])

newarr = np.diff(arr, n=2)

print(newarr)

>>>
[ 5 -30]
```

Returns: [5 -30] because: $15-10=5$, $25-15=10$, and $5-25=-20$ AND $10-5=5$ and $-20-10=-30$

Trigonometric Functions

NumPy provides the ufuncs `sin()`, `cos()` and `tan()` that take values in radians and produce the corresponding sin, cos and tan values.

Example:

Find sine value of $\pi/2$:

```
import numpy as np
```

```
x = np.sin(np.pi/2)
```

```
print(x)
```

```
>>>
```

```
1.0
```

Example:

Find sine values for all of the values in arr:

```
import numpy as np
```

```
arr = np.array([np.pi/2, np.pi/3, np.pi/4, np.pi/5])
```

```
x = np.sin(arr)
```

```
print(x)
```

```
>>>
```

```
[1.          0.8660254  0.70710678 0.58778525]
```

Convert Degrees into Radians

By default, trigonometric functions in NumPy use radians as inputs, but it is possible to convert radians to degrees and vice versa.

Example:

Convert all of the values in following array arr to radians:

```
import numpy as np

arr = np.array([90, 180, 270, 360])

x = np.deg2rad(arr)

print(x)

>>> [1.57079633 3.14159265 4.71238898 6.28318531]
```

Radians to Degrees

Example:

Convert all of the values in following array arr to degrees:

```
import numpy as np
```

```
arr = np.array([np.pi/2, np.pi, 1.5*np.pi, 2*np.pi])
```

```
x = np.rad2deg(arr)
```

```
print(x)
```

```
>>>
```

```
[ 90. 180. 270. 360.]
```

Finding Angles

Finding angles from values of sine, cos, tan. E.g. sin, cos and tan inverse (arcsin, arccos, arctan).

NumPy provides ufuncs `arcsin()`, `arccos()` and `arctan()` that produce radian values for the corresponding sin, cos and tan values are given.

Example:

Find the angle of 1.0:

```
import numpy as np
```

```
x = np.arcsin(1.0)
```

```
print(x)
```

```
>>>
```

```
1.5707963267948966
```

Hypotenuse

Finding hypotenuse using Pythagoras theorem in NumPy.

NumPy provides the `hypot()` function that takes the base and perpendicular values and produces hypotenuse based on Pythagoras theorem.

Example:

Find the hypotenuse for 4 base and 3 perpendicular:

```
import numpy as np
```

```
base = 3
```

```
perp = 4
```

```
x = np.hypot(base, perp)
```

```
print(x)
```

```
>>>
```

```
5.0
```

Hyperbolic Functions

NumPy provides the ufuncs `sinh()`, `cosh()` and `tanh()` that take values in radians and produce the corresponding sinh, cosh and tanh values..

Example:

Find sinh value of $\pi/2$:

```
import numpy as np

x = np.sinh(np.pi/2)

print(x)

>>>
2.3012989023072947
```

Example:

Find cosh values for all of the values in arr:

```
import numpy as np

arr = np.array([np.pi/2, np.pi/3, np.pi/4, np.pi/5])

x = np.cosh(arr)

print(x)

>>>
[2.50917848 1.60028686 1.32460909 1.20397209]
```

Finding Angles

Finding angles from values of hyperbolic sine, cos, tan. E.g. sinh, cosh and tanh inverse (arcsinh, arccosh, arctanh). NumPy provides ufuncs `arcsinh()`, `arccosh()` and `arctanh()` that produce radian values for the corresponding sinh, cosh and tanh values given.

Example:

Find the angle for all of the tanh values in array:

```
import numpy as np

arr = np.array([0.1, 0.2, 0.5])

x = np.arctanh(arr)

print(x)

>>>
[0.10033535 0.20273255 0.54930614]
```

Linear algebra

NumPy 2D arrays can be used to represent matrices efficiently in Python. We will just quickly go through some of the main matrix operations available.

Vectors in python

In Python, a vector can be represented in many ways, the simplest being a regular Python list of numbers:

Example:

```
vector = [10.5, 5.2, 3.25, 7.0]
```

we plan to do quite a lot of scientific calculations, it is much better to use NumPy's ndarray, which provides a lot of convenient and optimized implementations of essential mathematical operations on vectors.

Example:

```
import numpy as np

vector = np.array([10.5, 5.2, 3.25, 7.0])

print(vector)
>>array([10.5 ,  5.2 ,  3.25,  7.  ])
```

The size of a vector can be obtained using the size attribute:

Example:

```
import numpy as np

vector = np.array([10.5, 5.2, 3.25, 7.0])

print(vector.size)
>>4
```

Plotting vectors

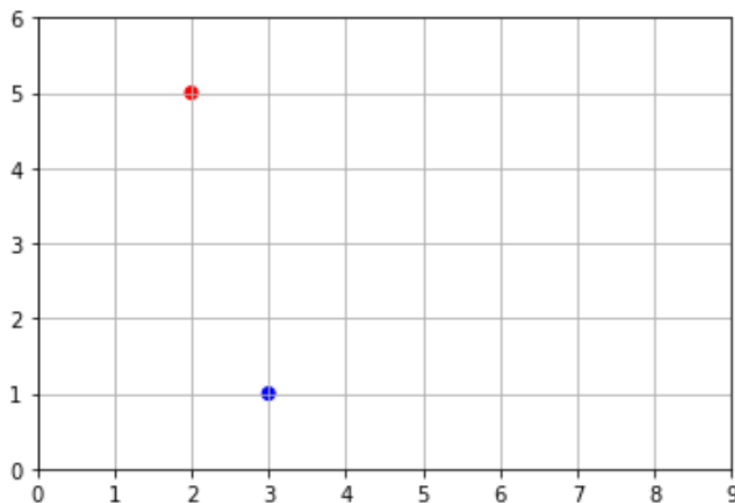
To plot vectors, we will use matplotlib, so let's start by importing it (for details about matplotlib

Example:

Let's create a couple of very simple 2D vectors to plot:

```
import matplotlib.pyplot as plt
u = np.array([2, 5])
v = np.array([3, 1])

x_coords, y_coords = zip(u, v)
plt.scatter(x_coords, y_coords, color=["r","b"])
plt.axis([0, 9, 0, 6])
plt.grid()
plt.show()
>>
```



Vectors can also be represented as arrows. Let's create a small convenience function to draw nice arrows:

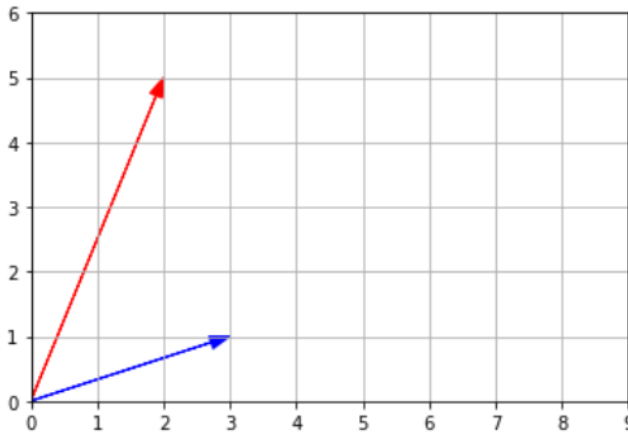
Example:

Let's create a couple of very simple 2D vectors to plot and represent as arrows:

```
import matplotlib.pyplot as plt
u = np.array([2, 5])
v = np.array([3, 1])

def plot_vector2d(vector2d, origin=[0, 0], **options):
    return plt.arrow(origin[0], origin[1], vector2d[0], vector2d[1],
                      head_width=0.2, head_length=0.3,
                      length_includes_head=True,
                      **options)

x_coords, y_coords = zip(u, v)
plt.scatter(x_coords, y_coords, color=["r", "b"])
plt.axis([0, 9, 0, 6])
plt.grid()
plt.show()
>>
```



Norm

```
u = np.array([2, 5])
v = np.array([3, 1])
```

The norm of a vector $\mathbf{u}$, noted $\|\mathbf{u}\|$, is a measure of the length (a.k.a. the magnitude) of $\mathbf{u}$. There are multiple possible norms, but the most common one (and the only one we will discuss here) is the Euclidian norm, which is defined as:

$$\|\mathbf{u}\| = \sqrt{\sum_i \mathbf{u}_i^2}$$

That's the square root of the sum of all the squares of the components of $\mathbf{u}$. We could implement this easily in pure python, recalling that $\sqrt{x} = x^{\frac{1}{2}}$

```
In [13]: def vector_norm(vector):
          squares = [element**2 for element in vector]
          return sum(squares)**0.5

          print("||", u, "|| =")
          vector_norm(u)
```

|| [2 5] || =

```
Out[13]: 5.385164807134504
```

However, it is much more efficient to use NumPy's `norm` function, available in the `linalg` (**Linear Algebra**) module:

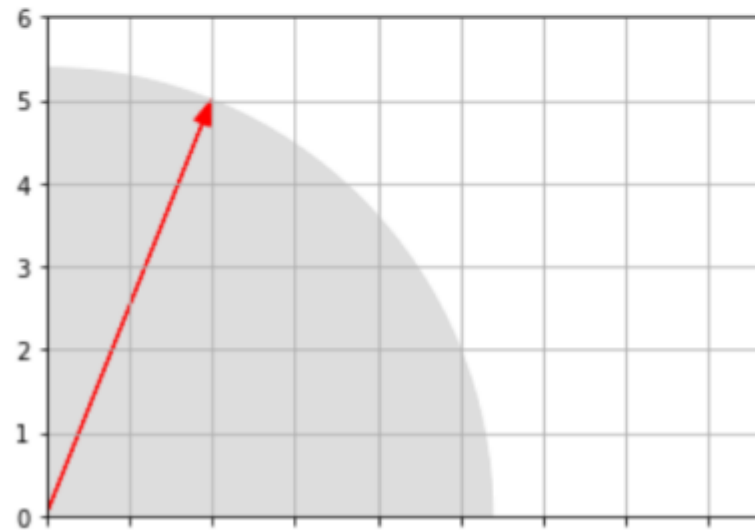
```
In [14]: import numpy.linalg as LA

          LA.norm(u)
```

```
Out[14]: 5.385164807134504
```

Let's plot a little diagram to confirm that the length of vector $\mathbf{u}$ is indeed ≈ 5.4 :

```
In [15]: radius = LA.norm(u)
plt.gca().add_artist(plt.Circle((0,0), radius, color="#DDDDDD"))
plot_vector2d(u, color="red")
plt.axis([0, 8.7, 0, 6])
plt.gca().set_aspect("equal")
plt.grid()
plt.show()
```



Addition

Vectors of same size can be added together. Addition is performed *elementwise*:

```
u = np.array([2, 5])  
v = np.array([3, 1])
```

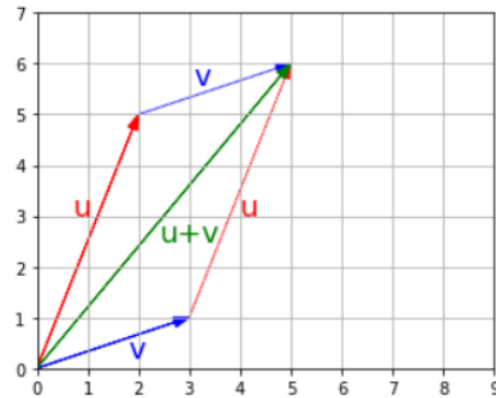
```
print(" ", u)  
print("+", v)  
print("-"*10)  
u + v
```

```
  [2 5]  
+ [3 1]  
-----
```

```
array([5, 6])
```

Let's look at what vector addition looks like graphically:

```
plot_vector2d(u, color="r")
plot_vector2d(v, color="b")
plot_vector2d(v, origin=u, color="b", linestyle="dotted")
plot_vector2d(u, origin=v, color="r", linestyle="dotted")
plot_vector2d(u+v, color="g")
plt.axis([0, 9, 0, 7])
plt.gca().set_aspect("equal")
plt.text(0.7, 3, "u", color="r", fontsize=18)
plt.text(4, 3, "u", color="r", fontsize=18)
plt.text(1.8, 0.2, "v", color="b", fontsize=18)
plt.text(3.1, 5.6, "v", color="b", fontsize=18)
plt.text(2.4, 2.5, "u+v", color="g", fontsize=18)
plt.grid()
plt.show()
```



Vector addition is **commutative**, meaning that $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$. You can see it on the previous image: following $\mathbf{u}$ then $\mathbf{v}$ leads to the same point as following $\mathbf{v}$ then $\mathbf{u}$.

Vector addition is also **associative**, meaning that $\mathbf{u} + (\mathbf{v} + \mathbf{w}) = (\mathbf{u} + \mathbf{v}) + \mathbf{w}$.

If you have a shape defined by a number of points (vectors), and you add a vector $\mathbf{v}$ to all of these points, then the whole shape gets shifted by $\mathbf{v}$.

```
t1 = np.array([2, 0.25])
t2 = np.array([2.5, 3.5])
t3 = np.array([1, 2])

x_coords, y_coords = zip(t1, t2, t3, t1)
plt.plot(x_coords, y_coords, "c--", x_coords, y_coords, "co")

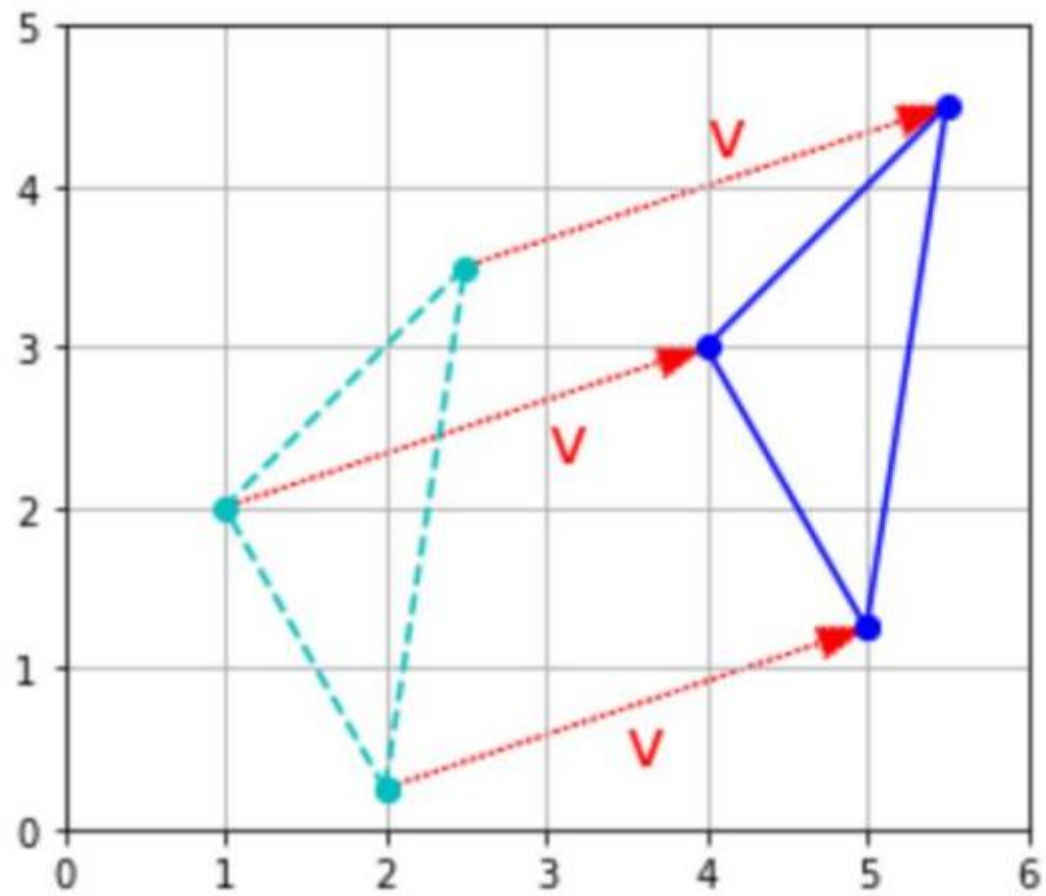
plot_vector2d(v, t1, color="r", linestyle=":")
plot_vector2d(v, t2, color="r", linestyle=":")
plot_vector2d(v, t3, color="r", linestyle=":")

t1b = t1 + v
t2b = t2 + v
t3b = t3 + v

x_coords_b, y_coords_b = zip(t1b, t2b, t3b, t1b)
plt.plot(x_coords_b, y_coords_b, "b-", x_coords_b, y_coords_b, "bo")

plt.text(4, 4.2, "v", color="r", fontsize=18)
plt.text(3, 2.3, "v", color="r", fontsize=18)
plt.text(3.5, 0.4, "v", color="r", fontsize=18)

plt.axis([0, 6, 0, 5])
plt.gca().set_aspect("equal")
plt.grid()
plt.show()
```



Finally, subtracting a vector is like adding the opposite vector.

Multiplication by a scalar

Vectors can be multiplied by scalars. All elements in the vector are multiplied by that number, for example:

```
print("1.5 *", u, "=")
```

```
1.5 * u
```

```
1.5 * [2 5] = array([3. , 7.5])
```

Graphically, scalar multiplication changes a figure's scale, hence the name scalar. The distance from the origin (the point at coordinates equal to zero) is multiplied by the scalar. For example, let's scale up by a factor of $k = 2.5$:

```
k = 2.5
t1c = k * t1
t2c = k * t2
t3c = k * t3

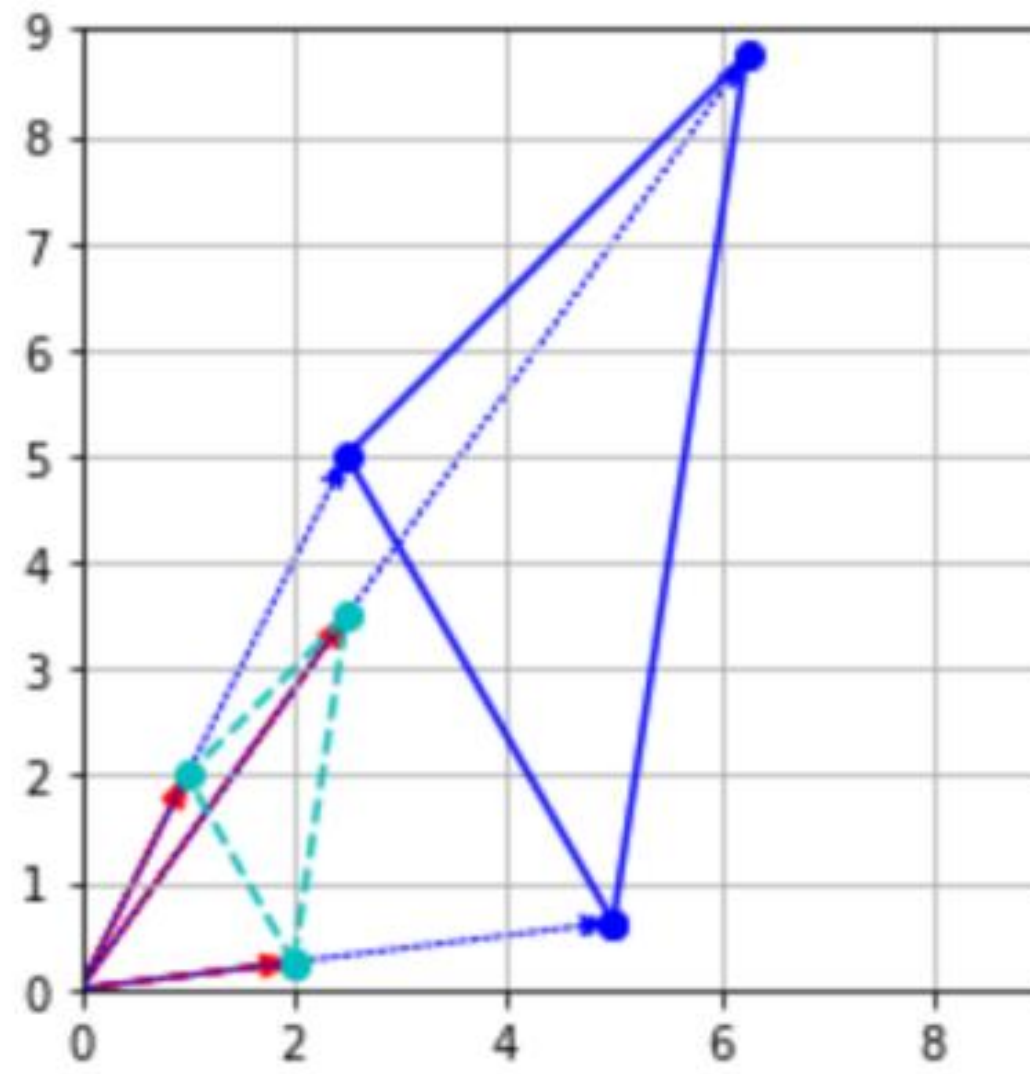
plt.plot(x_coors, y_coors, "c--", x_coors, y_coors, "co")

plot_vector2d(t1, color="r")
plot_vector2d(t2, color="r")
plot_vector2d(t3, color="r")

x_coors_c, y_coors_c = zip(t1c, t2c, t3c, t1c)
plt.plot(x_coors_c, y_coors_c, "b-", x_coors_c, y_coors_c, "bo")

plot_vector2d(k * t1, color="b", linestyle=":")
plot_vector2d(k * t2, color="b", linestyle=":")
plot_vector2d(k * t3, color="b", linestyle=":")

plt.axis([0, 9, 0, 9])
plt.gca().set_aspect("equal")
plt.grid()
plt.show()
```



As you might guess, dividing a vector by a scalar is equivalent to multiplying by its multiplicative inverse (reciprocal):

$$\frac{\mathbf{u}}{\lambda} = \frac{1}{\lambda} \times \mathbf{u}$$

Scalar multiplication is **commutative**: $\lambda \times \mathbf{u} = \mathbf{u} \times \lambda$.

It is also **associative**: $\lambda_1 \times (\lambda_2 \times \mathbf{u}) = (\lambda_1 \times \lambda_2) \times \mathbf{u}$.

Finally, it is **distributive** over addition of vectors: $\lambda \times (\mathbf{u} + \mathbf{v}) = \lambda \times \mathbf{u} + \lambda \times \mathbf{v}$.

Zero, unit and normalised vectors

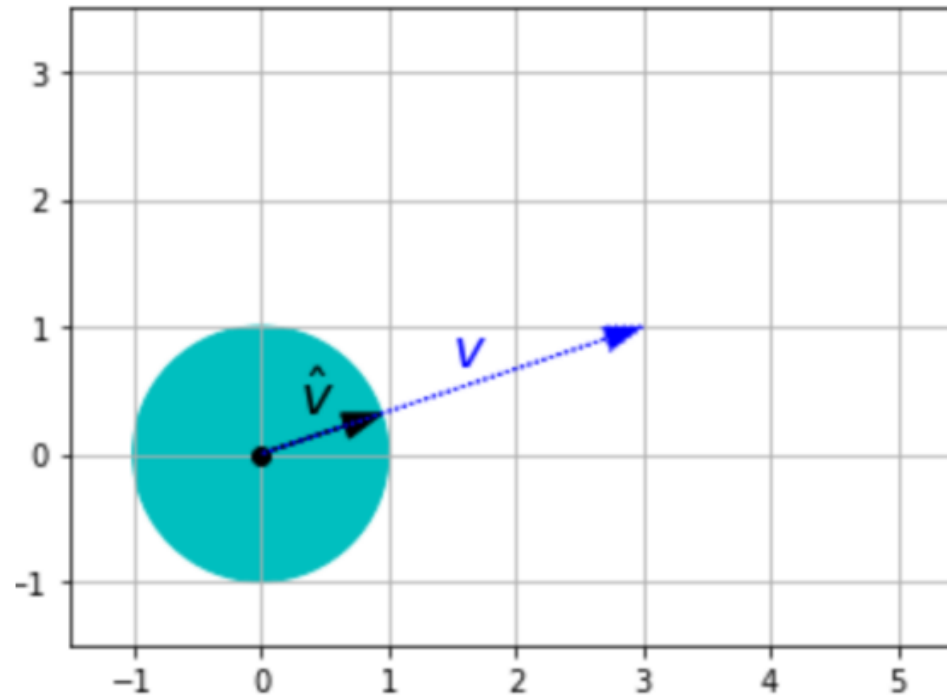
- A **zero-vector** is a vector full of 0s.
- A **unit vector** is a vector with a norm equal to 1.
- The **normalized vector** of a non-null vector $\mathbf{v}$, noted $\hat{\mathbf{v}}$, is the unit vector that points in the same direction as $\mathbf{v}$. It is equal to:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|}$$

```

plt.gca().add_artist(plt.Circle((0, 0), 1, color='c'))
plt.plot(0, 0, "ko")
plot_vector2d(v / LA.norm(v), color="k", zorder=10)
plot_vector2d(v, color="b", linestyle=":", zorder=15)
plt.text(0.3, 0.3, r"$\hat{v}$", color="k", fontsize=18)
plt.text(1.5, 0.7, "$v$", color="b", fontsize=18)
plt.axis([-1.5, 5.5, -1.5, 3.5])
plt.gca().set_aspect("equal")
plt.grid()
plt.show()

```



Dot product

Definition

The dot product (also called *scalar product* or *inner product* in the context of the Euclidian space) of two vectors $\mathbf{u}$ and $\mathbf{v}$ is a useful operation that comes up fairly often in linear algebra. It is noted $\mathbf{u} \cdot \mathbf{v}$, or sometimes $\langle \mathbf{u} | \mathbf{v} \rangle$ or $(\mathbf{u} | \mathbf{v})$, and it is defined as:

$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \times \|\mathbf{v}\| \times \cos(\theta)$$

where θ is the angle between $\mathbf{u}$ and $\mathbf{v}$.

Another way to calculate the dot product is:

$$\mathbf{u} \cdot \mathbf{v} = \sum_i \mathbf{u}_i \times \mathbf{v}_i$$

In Python, to implement the dot product is pretty simple to implement:

```
def dot_product(v1, v2):  
    return sum(v1i * v2i for v1i, v2i in zip(v1, v2))  
  
dot_product(u, v)
```

11

But a much more efficient implementation is provided by NumPy with the `np.dot()` function:

```
np.dot(u, v)
```

11

Equivalently, you can use the `dot` method of `ndarrays`:

```
u.dot(v)
```

11

❖ **Note:** the `*` operator will perform an elementwise multiplication, NOT a dot product.

```
: print(" ", u)  
  print("* ", v, "(NOT a dot product)")  
  print("-"*10)
```

```
u * v
```

```
    [2 5]  
*   [3 1] (NOT a dot product)  
-----  
: array([6, 5])
```

Matrices

A matrix is a rectangular array of scalars (i.e. any number: integer, real or complex) arranged in rows and columns, for example:

$$\begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{bmatrix}$$

You can also think of a matrix as a list of vectors: the previous matrix contains either 2 horizontal 3D vectors or 3 vertical 2D vectors.

Matrices are convenient and very efficient to run operations on many vectors at a time. We will also see that they are great at representing and performing linear transformations such rotations, translations and scaling.

Matrices in python

In python, a matrix can be represented in various ways. The simplest is just a list of python lists:

```
[  
    [10, 20, 30],  
    [40, 50, 60]  
]
```

```
[[10, 20, 30], [40, 50, 60]]
```

A much more efficient way is to use the NumPy library which provides optimized implementations of many matrix operations:

```
A = np.array(  
    [10, 20, 30],  
    [40, 50, 60]  
)  
A
```

```
array([[10, 20, 30],  
       [40, 50, 60]])
```

Conventionally, matrices are denoted by uppercase letters. For the purposes of this tutorial, we will be using NumPy arrays (ndarray type) to represent matrices.

Size

The size of a matrix is determined by the number of rows and columns it has. It is usually noted as (number of rows) x (number of columns). For instance, the matrix shown above is an example of a 2x3 matrix, which means it has 2 rows and 3 columns. However, be careful as a 3x2 matrix would have 3 rows and 2 columns.

To find out the size of a matrix in NumPy:

```
A.shape
```

```
(2, 3)
```

- ❖ **Note:** the size attribute represents the number of elements in the ndarray, not the matrix's size.

```
A.size
```

```
6
```

Adding matrices

If two matrices, **Q** and **R**, have the same size **m×n**, they can be added together. Addition is performed elementwise: the result is also an **m×n** matrix **S** where each element is the sum of the elements at the corresponding position: **S_{i,j} = Q_{i,j} + R_{i,j}**

$$S = \begin{bmatrix} Q_{11} + R_{11} & Q_{12} + R_{12} & Q_{13} + R_{13} & \cdots & Q_{1n} + R_{1n} \\ Q_{21} + R_{21} & Q_{22} + R_{22} & Q_{23} + R_{23} & \cdots & Q_{2n} + R_{2n} \\ Q_{31} + R_{31} & Q_{32} + R_{32} & Q_{33} + R_{33} & \cdots & Q_{3n} + R_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ Q_{m1} + R_{m1} & Q_{m2} + R_{m2} & Q_{m3} + R_{m3} & \cdots & Q_{mn} + R_{mn} \end{bmatrix}$$

For example, let's create a 2×3 matrix B and compute $A + B$:

```
B = np.array([[1, 2, 3], [4, 5, 6]])  
B
```

```
array([[1, 2, 3],  
       [4, 5, 6]])
```

```
A
```

```
array([[10, 20, 30],  
       [40, 50, 60]])
```

```
A + B
```

```
array([[11, 22, 33],  
       [44, 55, 66]])
```

Addition is *commutative*, meaning that $A + B = B + A$:

```
B + A
```

```
array([[11, 22, 33],  
       [44, 55, 66]])
```

It is also *associative*, meaning that $A + (B + C) = (A + B) + C$:

```
C = np.array([[100, 200, 300], [400, 500, 600]])
```

```
A + (B + C)
```

```
array([[111, 222, 333],  
       [444, 555, 666]])
```

```
(A + B) + C
```

```
array([[111, 222, 333],  
       [444, 555, 666]])
```

Scalar multiplication

A matrix **M** can be multiplied by a scalar **λ**. The result is noted **λM**, and it is a matrix of the same size as **M** with all elements multiplied by **λ**:

$$\lambda M = \begin{bmatrix} \lambda \times M_{11} & \lambda \times M_{12} & \lambda \times M_{13} & \cdots & \lambda \times M_{1n} \\ \lambda \times M_{21} & \lambda \times M_{22} & \lambda \times M_{23} & \cdots & \lambda \times M_{2n} \\ \lambda \times M_{31} & \lambda \times M_{32} & \lambda \times M_{33} & \cdots & \lambda \times M_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \lambda \times M_{m1} & \lambda \times M_{m2} & \lambda \times M_{m3} & \cdots & \lambda \times M_{mn} \end{bmatrix}$$

A more concise way of writing this is:

$$(\lambda M)_{i,j} = \lambda(M)_{i,j}$$

In NumPy, simply use the `*` operator to multiply a matrix by a scalar. For example:

```
2 * A
```

```
array([[ 20,  40,  60],  
       [ 80, 100, 120]])
```

Scalar multiplication is also defined on the right-hand side, and gives the same result: $M\lambda = \lambda M$. For example:

```
A * 2
```

```
array([[ 20,  40,  60],  
       [ 80, 100, 120]])
```

This makes scalar multiplication **commutative**.

It is also **associative**, meaning that $\alpha(\beta M) = (\alpha \times \beta)M$, where α and β are scalars. For example:

```
2 * (3 * A)
```

```
array([[ 60, 120, 180],  
       [240, 300, 360]])
```

```
(2 * 3) * A
```

```
array([[ 60, 120, 180],  
       [240, 300, 360]])
```

Finally, it is **distributive over addition** of matrices, meaning that $\lambda(Q + R) = \lambda Q + \lambda R$:

```
2 * (A + B)
```

```
array([[ 22,  44,  66],  
       [ 88, 110, 132]])
```

```
2 * A + 2 * B
```

```
array([[ 22,  44,  66],  
       [ 88, 110, 132]])
```

Matrix multiplication

Let's multiply two matrices in NumPy, using ndarray's `np.matmul()` function:

$$E = AD = \begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{bmatrix} \begin{bmatrix} 2 & 3 & 5 & 7 \\ 11 & 13 & 17 & 19 \\ 23 & 29 & 31 & 37 \end{bmatrix} = \begin{bmatrix} 930 & 1160 & 1320 & 1560 \\ 2010 & 2510 & 2910 & 3450 \end{bmatrix}$$

```
D = np.array([
    [ 2,  3,  5,  7],
    [11, 13, 17, 19],
    [23, 29, 31, 37]
])
E = np.matmul(A, D)
E
```

```
array([[ 930, 1160, 1320, 1560],
       [2010, 2510, 2910, 3450]])
```

A @ D

```
array([[ 930, 1160, 1320, 1560],
       [2010, 2510, 2910, 3450]])
```

The `@` operator also works for vectors. `u @ v` computes the dot product of `u` and `v`:

u @ v

Matrix transpose

The transpose of a matrix $\mathbf{M}$ is a matrix noted $\mathbf{M}^T$ such that the i^{th} row in $\mathbf{M}^T$ is equal to the i^{th} column in $\mathbf{M}$:

$$A^T = \begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{bmatrix}^T = \begin{bmatrix} 10 & 40 \\ 20 & 50 \\ 30 & 60 \end{bmatrix}$$

In other words, $(A^T)_{i,j} = A_{j,i}$

Obviously, if M is an $m \times n$ matrix, then M^T is an $n \times m$ matrix.

Note: there are a few other notations, such as M^t , M_t , or tM .

In NumPy, a matrix's transpose can be obtained simply using the `T` attribute:

```
A
```

```
array([[10, 20, 30],  
       [40, 50, 60]])
```

```
A.T
```

```
array([[10, 40],  
       [20, 50],  
       [30, 60]])
```

As you might expect, transposing a matrix twice returns the original matrix:

```
A.T.T
```

```
array([[10, 20, 30],  
       [40, 50, 60]])
```

Transposition is distributive over addition of matrices, meaning that $(Q + R)^T = Q^T + R^T$. For example:

```
(A + B).T
```

```
array([[11, 44],  
       [22, 55],  
       [33, 66]])
```

```
A.T + B.T
```

```
array([[11, 44],  
       [22, 55],  
       [33, 66]])
```

Moreover, $(Q \cdot R)^T = R^T \cdot Q^T$. Note that the order is reversed. For example:

```
(A @ D).T
```

```
array([[ 930, 2010],  
       [1160, 2510],  
       [1320, 2910],  
       [1560, 3450]])
```

```
D.T @ A.T
```

```
array([[ 930, 2010],  
       [1160, 2510],  
       [1320, 2910],  
       [1560, 3450]])
```

Matrix Inverse

The inverse of a matrix $\mathbf{A}$ is denoted as $\mathbf{A}^{-1}$ and is defined such that $\mathbf{A} * \mathbf{A}^{-1} = \mathbf{A}^{-1} * \mathbf{A} = \mathbf{I}$, where $\mathbf{I}$ is the identity matrix of the same dimension as $\mathbf{A}$. The identity matrix is a square matrix with ones on the diagonal and zeros elsewhere.

Conditions for the Existence of an Inverse

Not all matrices have inverses. A matrix must be square (the same number of rows and columns) and have a non-zero determinant to be invertible. This matrix type is known as 'non-singular' or 'invertible.'

How to Compute the Inverse using NumPy

NumPy provides the '`np.linalg.inv()`' function to compute the inverse of a given square matrix. Here how to use it:

Step 1: Import NumPy.

Step 2: Create a square matrix 'A'.

Step 3: Use '`np.linalg.inv(A)`' to compute the inverse of the matrix.

Example:

```
import numpy as np

# Step 2: Create a square matrix A
A = np.array([[1, 2], [3, 4]])

# Step 3: Compute the inverse of matrix A
A_inv = np.linalg.inv(A)

print("A:", A)

print("A\u207B\u00B9:", A_inv)
```

```
A: [[1 2]
     [3 4]]
A-1: [[-2.  1.]
       [ 1.5 -0.5]]
```

```
# Verify the result
identity_matrix = np.dot(A, A_inv)

# The identity_matrix should be close to [[1, 0], [0, 1]]
print(identity_matrix)

[[1.00000000e+00  0.00000000e+00]
 [8.8817842e-16  1.00000000e+00]]
```

- ❖ **Note:** When you try to invert a non-invertible (singular) matrix, NumPy will raise a 'LinAlgError'. You can handle this with a try-except block:

Thank you